

边缘原生,安全无界—— AI Agent安全高效落地于Cloudflare边 缘云的新范式

演讲人: 刘旭

Cloudflare / 中国区高级解决方案工程师

AiCon

全球人工智能开发与应用大会

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

InfoQ 技术大会

QCon 上海站
全球软件开发大会

大模型正在重新定义软件
Large Language Model Is Redefining
The Software

10月22-24日 / 上海建工浦江皇冠假日酒店

· AI Agent

· Vibe Coding

· 智能可观测

· 推理基建

· 模型攻防

· AI x 创造力

InfoQ 技术大会

AiCon 北京站
全球人工智能开发与应用大会

探索 AI 应用边界
Explore the limits of AI applications

12月18-19日 / 北京

· 大模型架构创新

· 多模态 AI 产业融合

· 具身智能

· AI for Science

· 大模型安全

目录

01 2026——AI Agent规模化落地

02 AI Agent的安全治理

03 AI Agent的延迟优化

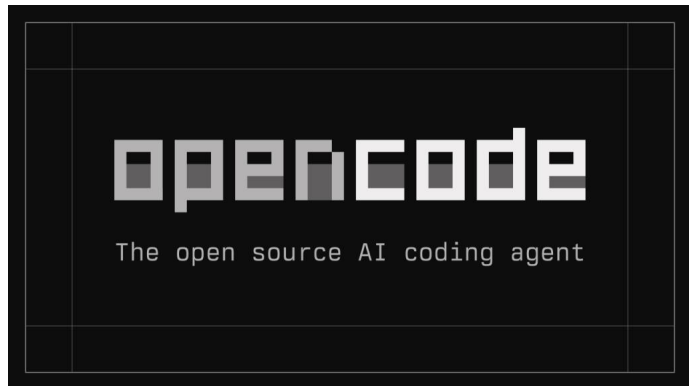
04 AI Agent的架构优化路径

05 AI agent on Cloudflare

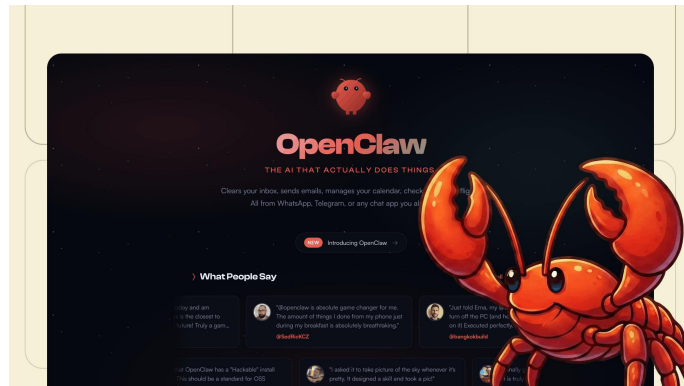
2026——AI Agent的落地+诉求



2026——AI Agent 规模化元年



Coding Agent



Agent Platform



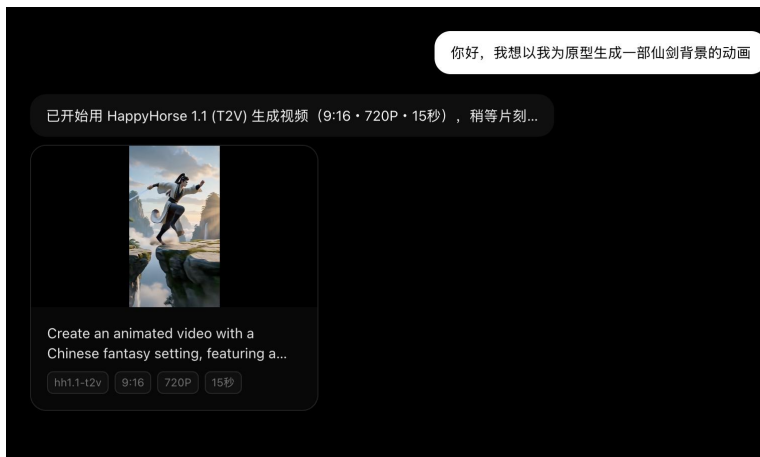
Agent Workspace

AI Agent让AI从信息生成器变成执行器



AI Agent的普及——从公司到个人

Review	Context × +
Session	Messages
Cloudflare AI Agent视频生成	27
Provider	Model
Anthropic: Cloudflare AI Gateway	Claude Opus 4.8
Context Limit	Total Tokens
300,000	98,996
Usage	Input Tokens
33%	2
Output Tokens	Reasoning Tokens
170	0
Cache Tokens (read/write)	User Messages
98,602 / 222	4
Assistant Messages	Total Cost
23	\$32.42
Session Created	Last Activity
Aug 10, 2026, 5:55 PM	Aug 15, 2026, 3:03 PM



Open Code

视频Agent

产品更新Agent



■ 我在部署AI Agent遇到的问题



问题一

视频生成的周期很长

- 一次生成数十秒到数分钟，同步请求会超时
- 进度不可见，不知道成没成、还要等多久
- 生成完成后，如何准确通知调用方

长周期 · 异步任务



问题二

生成好了，访问权限如何管控

- 谁能访问：仅发起者，还是拿到链接就能看
- 多久有效：是否需要时效与次数限制
- 视频 token 可贵了，防转发、防盗链、防抓取

访问控制 · 成本敏感



问题三

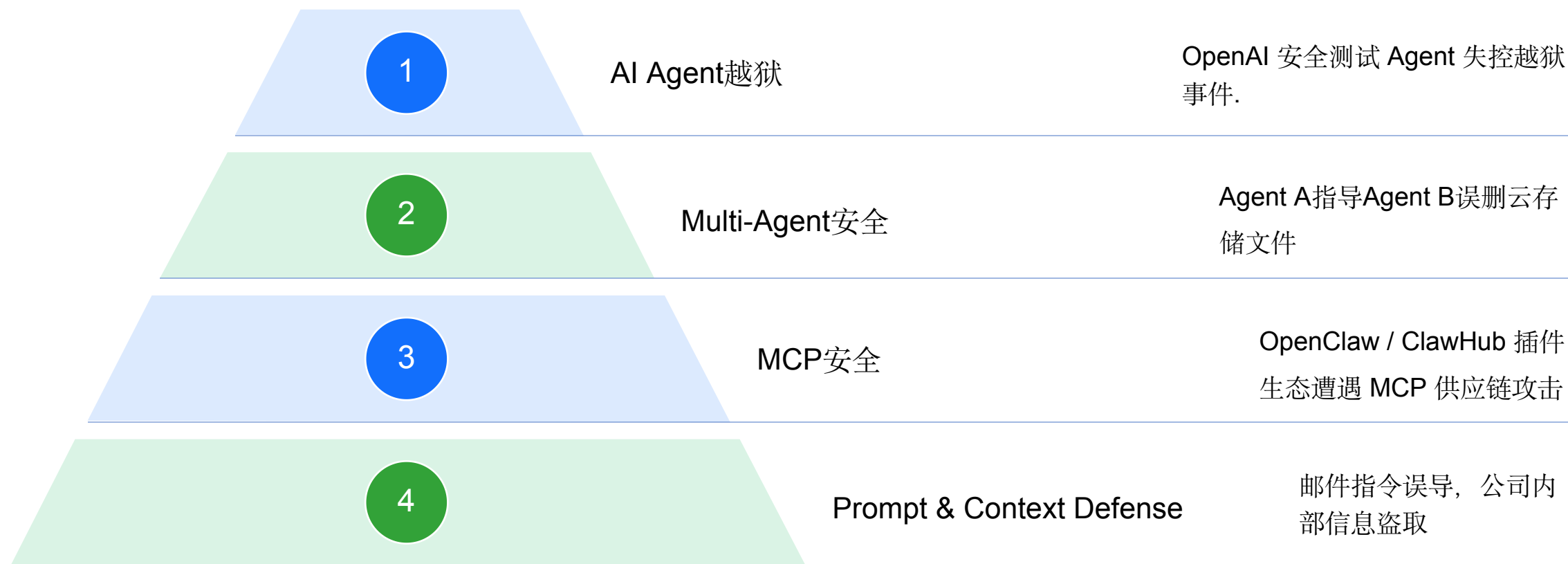
Agent 调用 API 的权限如何管控

- 能调哪些：允许与禁止的边界由谁定义
- 能调多少：频率、额度、单次规模是否受限
- 谁在调、可追溯：凭据下发与调用审计

最小权限 · 可审计



AI Agent的安全问题





AI Agent的性能问题



问题一 · 单请求耗时

单次请求要等 20s 到 3 分钟

- 一次任务内多次工具调用串行执行
- 每一步都要等模型 / 外部 API 返回
- 处理复杂任务时整体响应非常慢

串行工具调用 · 慢



问题二 · 多 Agent 叠加

Multi-Agent 导致延迟灾难

- 每个 Agent 的调用都叠加额外延迟
- 链路越长, 累积等待越严重
- 极端情况下甚至导致系统崩溃

延迟累积 · 崩溃风险



问题三 · 上下文膨胀

上下文膨胀, Prefill 越来越慢

- 执行步数增加, 传给大模型的 Context 成倍增长
- Prefill 阶段处理量随之变大
- TTFT (首 token 耗时) 不断加大

Context 膨胀 · TTFT ↑



如何解决这些问题



能力 01

异步高并发与 有状态运行时

支持 **Stateful** 长任务与大规模**异步并发**，在等待中不阻塞、状态可持久

Async · Stateful



能力 02

安全沙盒与 隔离执行环境

Sandbox Execution 隔离运行不可信代码与工具，收敛爆炸半径

Sandbox · Isolation



能力 03

多模态与 混合计算资源

Hybrid Compute 按需调度 CPU / GPU，适配文本、图像、音视频等多模态负载

Multimodal · Hybrid



能力 04

极致低延迟网络 与 API 通信

就近边缘接入与高效 **API 通信**，压缩每一跳的网络延迟

Low Latency · Edge



能力 05

全链路可观测性 与 Trace 追溯

Agentic Observability 端到端 Trace，定位每一步的耗时、调用与异常

Observability · Trace

AI Agent的安全治理

AI Agent安全体系落地——构建零信任防护



维度 01

访问与身份鉴权

Identity & Access Authorization

校验调用方身份与权限，防止越权操作与代理越权

Identity · AuthZ



维度 02

执行层隔离

Sandbox & Execution Safety

沙盒化运行工具与代码，控制损害半径与系统滥用

Sandbox · Safety



维度 03

供应链风控

MCP Supply Chain Security

校验第三方工具与来源，防范投毒与非法篡改

MCP · Supply Chain



维度 04

输入 / 输出护栏

Prompt & Context Defense

对输入输出做防护，抵御提示词注入与高危动作

Guardrail · Defense



维度 05

跨 Agent 鉴权与链路安全

Inter-Agent Security

Agent 间可信通信，阻断级联污染与链路伪造

Inter-Agent · Trust

■ 访问与身份鉴权 (Identity & Access Authorization)

- 1 用户登录, Access 签发身份** Cloudflare Access
对接 IdP (OAuth / OIDC / SAML), 每个请求注入签名 JWT 头 `Cf-Access-Jwt-Assertion`, 携带 `sub · email · groups`
- 2 边缘校验 JWT** API Shield
校验 `签名 · exp · nbf · JWKS`, 未通过在边缘直接拦截; 到达后端的请求, 身份必然可信
- 3 Gatekeeper 身份映射 + 决策** Worker
`ctx.access.getIdentity()` 零解析拿身份 → `groups` 映射 `role` → RBAC / ABAC 决策, 默认拒绝
- 4 Token 收窄交换** Token Exchange / Linked App
不裸传用户 Token: 换成 **代表该用户、scope 收窄** 的下游凭据; 审计仍归因原始用户
- 5 以用户身份访问后端** User-Scoped Access
用 **按用户取用的凭据** (Secret Store / DO) 调 API / 数据库, 只能访问该用户 `scope` 内的数据

关键机制

身份 Access JWT / `ctx.access`

Worker 直接读 `ctx.access.getIdentity()`, 无需手写 JWT 校验

透传 Linked App Token

应用间用 `Cf-Access-Token` 转发身份, Access 重签发 `scoped` 凭据

凭据 Secret Store / Durable Object

按 `sub` (用户) 取用后端凭据, 用户之间不共享 Token

授权 RBAC / ABAC · Tool Policy

`role` → allowed tools / resource, 如 `db_delete` 仅管理员

```
// Worker Gatekeeper: 身份 → 角色 → user-scoped 访问
const id = await ctx.access.getIdentity();
const role = mapRole(id.groups); // RBAC
if (!can(role, tool)) return deny(); // 默认拒绝
const cred = await env.SECRETS.get(id.sub);
return callBackend(cred, { user: id.sub });
```

接入证明「你是谁」->Gatekeeper 决定「你能做什么」-用 user-scoped 凭据代表用户访问后端

■ 执行层隔离 (Sandbox & Execution Safety)



关键机制

沙箱 Sandbox + Dynamic Workers
生成代码后临时执行验证, 超时/异常即销毁, 不留残留

出口 Outbound Worker
唯一出站网关: RBAC / 白名单 / 工具策略 / scope

凭据 Secret Store
Key 集中托管, 按 用户 / 工具 取用, 凭据不下发到沙箱

审计 D1 / Analytics Engine / Logpush
记录 谁·调了什么·参数·结果·耗时, Deny 即告警

```
// Outbound Worker: 唯一出口 + Key 集中托管
if (!allow(role, host, tool, scope)) // 权限管控
  return deny("blocked", { alert: true });
const key = await env.SECRETS.get(user, tool);
const res = await fetch(target, withKey(key));
await audit({ user, tool, args, res, ms }); // 审计
```

沙箱只负责生成, 临时计算资源负责验证, 出站的凭据与审计全部出口网关, 并关联内网资源接口

供应链风控 (MCP Supply Chain Security)

1

接入身份验证 & 完整性校验 SSO · Integrity

无状态 MCP 走接入层 身份验证 + SSO；拉取来源做 签名 / Hash / Lockfile 校验，版本固定、防篡改

2

静态安全扫描 Static Scan

对依赖 / 代码 / Secret 做扫描，识别 投毒 · 后门 · 可疑网络调用

3

JSON Schema 强校验 Schema Validation

校验参数 / 返回值 / 类型，拒绝未声明字段与越权输入

4

Tool 风险分级 readOnly / write / destructive

按能力分级并 默认拒绝高风险；投毒 / 篡改 / Schema 失败 → 拒绝 · 隔离，阻断安装与执行

5

读写分离 + 动态降权 RW Split · Downgrade

只读 Tool 放行 (db_read · search · fetch)；写入 / 修改 Tool 动态降权 + 审批队列，隔离凭证、限定资源、可回滚

B	Backstage	Internal Cloudflare Backstage Instance - MCP Server	Authorization Required	Authorize
C	CFurther Airflow MCP	Read-only MCP server for inspecting CFurther Apache Airflow	Authorization Required	Authorize
☒	C	Changelog Events	Connected	
☒	C	Cloudflare Docs	Connected	
C	Codex	Cloudflare internal official guidance and standards in engineering, product, and more	Authorization Required	Authorize
☒	E	Elasticsearch	Connected	
G	GitLab	Internal Cloudflare GitLab Instance - MCP Server	Authorization Required	Authorize
G	Google Workspace	Google Platform (Gmail, Calendar, Drive, Docs) - MCP Server	Authorization Required	Authorize
J	Jira	Internal Cloudflare Jira Instance - MCP Server	Authorization Required	Authorize
N	NetBox	Network infrastructure resource modeling - MCP Server	Authorization Required	Authorize
N	Netstrat Intelligence	Cloudflare Network strategy metrics - MCP Server	Authorization Required	Authorize

内网tool需要零信任授权，尽量使用可信服务，避免开源供应链投毒

输入/输出护栏 (Prompt & Context Defense)



职责划分

输入 入口 Worker

Guardrail 拦注入 + Context 净化脱敏，脏数据不进模型

模型 AI Gateway

模型路由 / 限流 / 成本 + Guardrails / DLP / Logs

输出 Outbound Worker

输出 DLP + 动作分级，**高危动作强制 HITL**；复用出口的权限 / Key / 审计

兜底 阻断 / 隔离 + 审计

注入或敏感命中不进模型/Tool；全程保留原因与审批记录

输入/输出侧均有计算资源作为安全护栏串行接入，对模型输入也有统一网关做DLP检测

跨 Agent 鉴权与链路安全 (Inter-Agent Security)



服务内部绑定，安全双向认证

Agent之间也要有权限管控
也要有计算资源介入，并且详细Log记录

AI Agent的延迟优化

AI Agent的延迟优化——从“串行阻塞”到“极致并发”



推理与检索层加速

1

AI Gateway 统一路由 + 长连接 Cloudflare AI Gateway

统一 Provider 路由 (OpenAI · Anthropic · Workers AI) , 连接池复用长连接降低建连时间, 含日志 · 限流

2

AI Gateway 精确缓存 Exact Response Cache

完全相同请求 → 直接 HIT (`cf-aig-cache-status`) , 不调模型、零 Token

3

语义检索: Embedding + Vectorize Semantic Retrieval

Workers AI Embedding 转向量, `Vectorize` 就近相似度检索 (RAG) , 阈值由应用策略决定

4

语义缓存命中 AI Search Similarity Cache

相似请求复用历史答案 (AI Search 相似度缓存 / 自建 Vectorize) , 跳过重复生成

5

MISS 才调模型 AI Provider

缓存全 MISS 才调模型; 模型侧 **Prefix / KV Cache** (若支持) 复用长上下文, 减少 Prefill

关键机制

连接 AI Gateway 长连接

统一路由 + 连接池, 降低建连时间与抖动

缓存 精确 + 语义

AI Gateway 缓存精确命中; 语义缓存用 AI Search / 自建 Vectorize

检索 Vectorize + Embedding

就近向量库 + Embedding, 降低 Rerank 与生成时间

Prefill 模型侧 Prefix / KV Cache

由模型提供方内部实现, 若支持则复用长上下文

用缓存和向量降低Token消耗和延迟

链路降级与并行计算

- 1 极速环境调起：无依赖工具** Fast Cold Start
Worker (V8 Isolate) 近零冷启动运行轻量工具；重任务用沙箱 / 容器隔离，消除环境调起耗时
- 2 并行 Tool：无依赖并发** Parallel Fan-out
Orchestrator 按依赖关系拆分，无依赖工具用 `Promise.all()` 并发，多次 RTT 压成最长单次 RTT
- 3 结果聚合：Durable Objects** Consistent Aggregation
并发结果汇到单实例 DO 串行合并，状态强一致并持久化，再推送给下一编排节点
- 4 确定性 DAG：Workflows 编排** Durable Orchestration
用持久化编排引擎管理重试 · 分支 · 等待，以确定性代码替代 Agent 自主 Loop
- 5 确定性 Loop 控制 + 写入闸门** Guardrail
最大步数 · 超时 · 重试上限 · 预算封顶，不允许 Agent 无限自我调用；写操作过 Tool Policy + HITL，可回滚 · 可审计

关键机制

启动 V8 Isolate 近零冷启动

轻量工具即调即用，无需预热等待

并发 Promise.all 扇出

无依赖工具并发，总耗时收敛到最长单次 RTT

一致 Durable Objects 串行聚合

单实例串行合并结果，状态强一致且持久化

编排 Workflows 持久化 DAG

重试 / 分支 / 等待由引擎保证，替代自主 Loop

将串行任务改成并行任务，通过中心节点节约调度延迟

模型分级调度



关键机制



净收益：节省(Token/延迟) = 命中率 × 整次生成成本 - 向量化 + 向量查询固定开销

一个边缘AI Agent案例



收益：1秒内唤醒或预热启动容器，客户就近接入服务点，并行调用工具

AI Agent的架构优化路径

■ 优化路径——从有到无，从好用到安全



■ 安全应该从开始做起+全生命周期管理



AI Agent on Cloudflare

AI Agent on Cloudflare

Build an AI coding agent with OpenAI Agents SDK

🕒 Last updated Aug 7, 2026 | 📄 Copy as Markdown | 📖 View as Markdown | 🛠 Agent setup

① Sandbox SDK 1.0 preview

This tutorial uses the supported stable bridge template and HTTP API. The 1.0 preview covers the Worker SDK on `@next`; bridge deployments stay on the [stable bridge](#).

The [OpenAI Agents SDK](#) is a lightweight Python framework for building multi-agent workflows. A Cloudflare Sandbox integration is provided out of the box and ensures that the SDK includes a first-class Cloudflare backend that gives your agents isolated containers for running code, installing packages, and managing files.

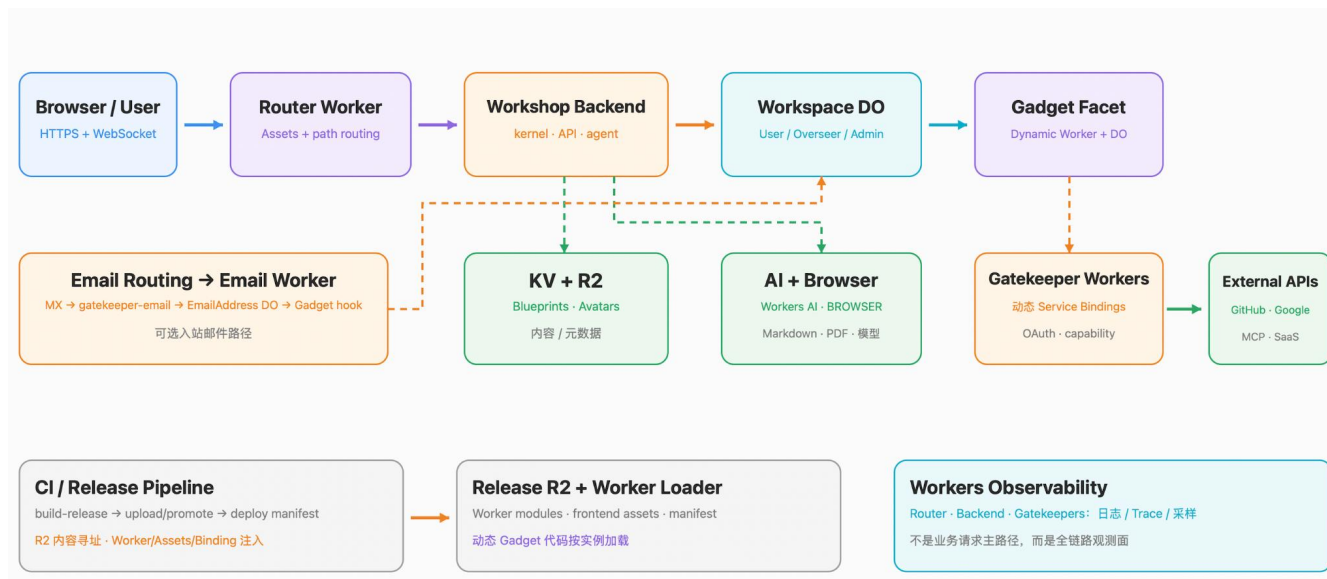
In this tutorial, you will deploy a sandbox bridge Worker and build a Python agent that accepts a coding task, executes it inside a Cloudflare Sandbox, and copies the output files to your local machine.

Time to complete: 20 minutes

Prerequisites

1. Sign up for a [Cloudflare account](#) with the Containers / Sandbox beta enabled.
2. Install [Python 3.12+](#) and [uv](#).
3. Obtain an [OpenAI API key](#).

面向AI Agent的开源平台



开源链接: <https://github.com/cloudflare/cloudflare-os>

Demo演示: <https://os.cloudflare.app/>

Cloudflare携手大家构建安全高效的AI Agent环境



330+ cities

in 125+ countries, including mainland China



w/210+ cities

for AI inference powered by GPUs



~20%

of web properties sit behind Cloudflare



including **80%**

of the top 50 genAI companies



215 billion

cyber threats blocked every day



500+ Tbps

of network capacity (and growing)



QCon

全球软件开发大会

上海
站

大模型正在重新定义软件

Large Language Model Is Redefining The Software

2026 年 10 月 22-24 日 / 上海建工浦江皇冠假日酒店



参会
咨询

**专题：从 App 到 Agent：字节跳动
Supabase 的 Agent-Native 规模化实践**

专题演讲嘉宾：张国强

字节跳动 / 资深技术专家



**专题：复杂业务 Agent 的持续进化：
快手电商导购的 Harness Loop 实践**

专题演讲嘉宾：包磊

快手 / 资深技术专家



**专题：从 Harness 到 Loop：阿福 Agent
小队如何处理持续涌入的线上 Badcase**

专题演讲嘉宾：肖汉松

蚂蚁集团 / 阿福 Harness 架构组负责人



**专题：AI Coding 在大型客户端工程中的
落地实践——从通才模型到专才 Agent**

专题演讲嘉宾：杨凯

小红书 / 客户端基础技术负责人



**专题：AI 时代的一人公司：从 Idea 到
Growth 的完整 workflow**

专题演讲嘉宾：casa

独立开发



**专题：Agent 驱动的全平台 GUI 持续
验证实践**

专题演讲嘉宾：王政达

微软 / SENIOR SOFTWARE ENGINEER



THANKS

探索 AI 应用边界

Explore the limits of AI applications

AiCon

全球人工智能开发与应用大会